ChatGPT

Sprint Management System (Spring Boot + React): Tech Stack & Architecture

- **Core Languages/Frameworks:** Spring Boot (Java 21+) for backend, React (TypeScript) for frontend [1]. Use Spring Web (REST API), Spring Data JPA (ORM), and Spring Security (auth) on the server; React with tools like Create React App or Next.js on the client [1] [2].
- **Database:** A relational DB (e.g. PostgreSQL or MySQL) for persistent data. Use JPA/Hibernate for data access [1]. Add Redis for caching frequently-read data [3].
- **State & UI:** Use React Query or Redux for client state management and server communication [2] [4]. UI libraries (Material-UI/Bootstrap) can speed up development.
- **Messaging/Events:** Include a message broker (RabbitMQ/Kafka) if needing async notifications or cross-service events (e.g. notifications of sprint updates).
- **Authentication/SSO:** Integrate OAuth2/OpenID Connect (e.g. Keycloak, Okta) for SSO [5] [6]. Spring Security's built-in JWT/OAuth2 support handles token validation [6].
- **DevOps & Infra:** Containerize both services (Docker). Orchestrate with Kubernetes or a cloud PaaS. Use a CI/CD tool (GitHub Actions, Jenkins, etc.) to automate build/test/deploy [7]. Set up environments for dev/test/prod.
- **Monitoring/Logging:** Spring Boot Actuator for health/metrics and ELK or Prometheus/Grafana for logs/metrics [8] [9]. Use structured logging (Logback/Log4j2) and distributed tracing (Spring Cloud Sleuth + Zipkin) for observability [10] [8].

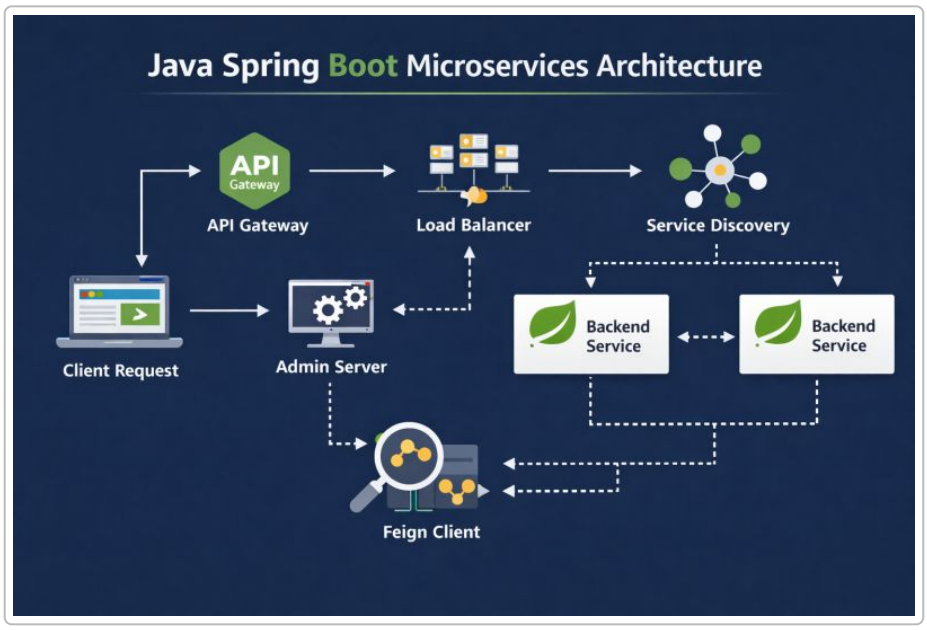

Figure: A typical Spring Boot microservices architecture with an API Gateway, load balancer, service registry, monitoring, and backend services [10].

In this design, the **React front-end** (served via CDN or static server) calls the **Spring Boot backend API** over HTTPS. The backend logic is modular: controllers → services → repositories. Service instances can scale horizontally behind a load balancer, registered via a service registry (e.g. Eureka) [10]. An API Gateway handles routing, security (authentication), and rate limiting. Each service uses a relational database; a caching layer (Redis) reduces DB load [3]. Actuator endpoints expose `/health` and `/metrics` [8], while a logging and tracing stack (Zipkin, ELK) captures system behavior [10] [11].

Database Schema (Normalized)

Design the data model to eliminate redundancy [12]. Key tables (examples):

- **Users:** `User(id, name, email, role, hashed_password, createdAt)` – stores account info.
- **Projects:** `Project(id, name, description, createdBy, createdAt)` – top-level agile project.
- **Sprints:** `Sprint(id, project_id→Project.id, name, startDate, endDate, status, velocity)` – linked to a Project.
- **Stories:** `Story(id, sprint_id→Sprint.id, title, description, storyPoints, status, priority, createdBy, createdAt)` – part of a Sprint.
- **StoryAssignments:** `Assignment(id, story_id→Story.id, user_id→User.id, skill, pointsAssigned, pointsCompleted)` – one row per user-task assignment. *(Removes unnormalized skill columns by normalizing per-assignment data.)*
- **Comments:** `Comment(id, story_id→Story.id, user_id→User.id, text, createdAt)`.
- **ActivityLog:** `Activity(id, user_id→User.id, actionType, targetType, targetId, timestamp)` – records changes (e.g. status moves) for audit trails.
- **Other:** e.g. `Role` or `Permission` tables if not covered by SSO; attachment/link tables if needed.

Each table's primary key is a single id. Foreign keys enforce relationships. For example, **StoryAssignments** replaces multiple skill-specific columns with rows, achieving 3NF (no redundant columns for each skill) [12] [13]. Normalizing like this **reduces redundancy and update anomalies** [12].

REST API Endpoints

Use RESTful routes following standard CRUD patterns [14]. Prefix endpoints with `/api/`. Example routes (methods + sample payloads):

- **Project APIs:**
- `GET /api/projects` – List all projects.
- `POST /api/projects` – Create project. *Request:* `{"name":"Alpha","description":"New project"}`.
- `GET /api/projects/{projectId}` – Get project details.
- `PATCH /api/projects/{projectId}` – Update project (partial). *Request:* `{"name":"Alpha V2"}`.
- `DELETE /api/projects/{projectId}` – Delete project.
- **Sprint APIs:**

- `POST /api/projects/{projectId}/sprints` – Add a sprint to a project. *Request:* `{"name":"Sprint 1","startDate":"2026-03-01","endDate":"2026-03-15"}`.
- `GET /api/projects/{projectId}/sprints` – List sprints for project.
- `GET /api/sprints/{sprintId}` – Get sprint details.
- `PATCH /api/sprints/{sprintId}` – Update sprint info.
- `DELETE /api/sprints/{sprintId}` – Delete sprint.
- **Story APIs:**
- `POST /api/sprints/{sprintId}/stories` – Create story in a sprint. *Request:* `{"title":"User login","storyPoints":5,"priority":"HIGH"}`.
- `GET /api/sprints/{sprintId}/stories` – List stories in sprint.
- `GET /api/stories/{storyId}` – Get story details.
- `PATCH /api/stories/{storyId}` – Update story (e.g. status change). *Request:* `{"status":"IN_PROGRESS"}`.
- `DELETE /api/stories/{storyId}` – Delete story.
- **Assignment/Progress APIs:**
- `POST /api/stories/{storyId}/assignments` – Assign a user to story. *Request:* `{"userId":7,"skill":"Java","pointsAssigned":3}`.
- `PATCH /api/stories/{storyId}/progress` – Update story progress (e.g. points completed). *Request:* `{"userId":7,"pointsCompleted":2}`.
- **Analytics APIs:**
- `GET /api/sprints/{sprintId}/burndown` – Get burndown data (JSON of date/points left).
- `GET /api/projects/{projectId}/velocity` – Get average velocity or historical stats.
- **Auth/User APIs:** If using SSO, much can be delegated. You may still have `GET /api/users/me` to fetch current user.

Each endpoint should return appropriate HTTP status (200/201/204 for success, 400/404 on error). Use DTOs to decouple entity fields and validate inputs. For example, creating a project might return `201 Created` with the new project or its ID. The pattern above follows common REST conventions [14]. Error details should follow a consistent JSON format.

Security, Performance & Audit Features

- **Authentication & Authorization:** Use **Spring Security** with OAuth2/OIDC for SSO (e.g. Keycloak/Okta) [5] [6]. Protect all API endpoints; e.g. require valid JWT for create/update calls. Apply **role-based access control** (e.g. only managers can end sprints). Enable CSRF protection and HTTPS for all connections.
- **Input Validation & Injection Defense:** Validate all inputs using Bean Validation (`@Valid`, `@Min/@Max`, etc.) [15]. Use JPA and parameterized queries to prevent SQL injection [15] (never concatenate SQL). Escape or sanitize output to prevent XSS [16]. Configure HTTP security headers (CSP, X-Frame-Options) to mitigate browser attacks [17].
- **Session/Security Hardening:** Set secure, HTTP-only cookies if using session tokens. Enforce short-lived tokens and refresh securely. Apply CORS rules to restrict API usage to your domains. Rate-limit requests at the API gateway to avoid abuse.
- **Logging & Audit:** Implement **audit logging** of key actions (e.g. user/story status changes). Use Spring interceptors or AOP to log `User X moved story Y to DONE at T` into an `ActivityLog` table [11]. This captures who did what for security/compliance [11]. Log all auth events and errors. Use **structured logs** (JSON) and forward to a central log store (ELK/Grafana).

- **Monitoring:** Enable Spring Boot **Actuator** and expose health/metrics endpoints [8]. Collect metrics (requests per second, latency, DB query times). Use alerts on error rate or slowdowns. Consider distributed tracing with Zipkin/Sleuth as part of microservices stack [10].
- **Performance Optimization:** Add **caching** for expensive reads (e.g. sprint stats, burndown data) [3]. Use database indexes on foreign keys and commonly queried fields (e.g. index `Sprint(project_id)` and `Story(sprint_id)`) [18]. Enable GZIP compression for API responses to reduce payloads [19]. Use asynchronous processing (`@Async`) for long tasks (email notifications, data exports) to avoid blocking [20]. In Spring Boot 3, consider virtual threads for high concurrency scenarios [21].
- **Resilience:** Use retry/circuit-breaker patterns (e.g. Spring Retry or Resilience4j) for external calls (if any). Implement proper exception handling (use `@ControllerAdvice`) to avoid leaking internals.

Implementation Roadmap (Weekly)

1. **Week 1 – Setup & Planning:** Finalize requirements and technology choices. Initialize Git repo and basic project scaffolding: generate Spring Boot app (Maven) and React app. Configure build tools (Maven/Gradle, Node). Set up database (local MySQL/Postgres) and run initial migrations (use Flyway/Liquibase for schema versioning). Create a high-level architecture diagram and ERD. (Milestone: "Env up, basic endpoints reachable".)

2. **Week 2 – Core Backend Models & APIs:** Design/implement core entities and repositories (User, Project, Sprint, Story, Assignment). Enforce relationships and constraints. Develop corresponding REST controllers with CRUD endpoints (use @RestController). Write unit tests for service methods (JUnit + Mockito). Set up Swagger/OpenAPI docs. (Milestone: "Basic project/sprint/story endpoints with tests".)

3. **Week 3 – Frontend UI & Integration:** Build React UI components for Projects and Sprints (e.g. project list, sprint form) and Story management. Use Axios/React Query to call backend endpoints. Integrate login (redirect to SSO) and fetch user info via token. Ensure CORS/auth integration. Implement project/sprint/story forms and list pages. (Milestone: "User can create projects/sprints and view them.")

4. **Week 4 – Progress Tracking & Analytics:** Add story assignment and progress features. Frontend: interfaces to assign stories and update progress. Backend: endpoints for `/assignments` and burndown data. Compute sprint burndown (backlog reduction) server-side and return JSON for charting. Implement simple chart on front (e.g. Chart.js). (Milestone: "Sprint progress trackable; burndown chart visible.")

5. **Week 5 – Security & Audit:** Integrate full auth flow: secure all APIs with Spring Security and OAuth2 (e.g. configure Keycloak). Implement roles (Admin, Manager, Developer). Add logging of actions (e.g. status changes) into an audit table [11]. Create an audit log viewer page or API. Begin end-to-end tests (Selenium/Cypress) for critical flows. (Milestone: "Authorization enforced; audit log captured.")

6. **Week 6 – Optimization & Extra Features:** Optimize queries and add caching for heavy endpoints [3]. Enable GZIP. Add background jobs (e.g. nightly summary emails or sprint closure tasks using Spring Scheduler or an async queue). Polish UI/UX: draggable story boards or Kanban view. Implement mobile responsiveness. (Milestone: "App performance tuned; extra features added.")

7. **Week 7 – CI/CD Pipeline & Testing:** Create CI/CD pipeline (e.g. GitHub Actions). Steps: lint & build front, run frontend tests; build backend JAR, run backend tests; build Docker images and push to registry. Deploy to staging (Heroku/GKE/ECS). Write integration tests (@SpringBootTest with Testcontainers) to run in pipeline. Use Cypress for UI E2E tests against deployed app. (Milestone: "Automatic build & deploy; tests passing.")

8. **Week 8 – Deployment & Documentation:** Finalize deployment (configure production DB, secrets, domain, HTTPS). Perform load testing (JMeter) and fix bottlenecks. Prepare README and system documentation (architecture, API reference, deployment steps). Optionally implement rollback strategy (blue/green or versioned deploy). (Milestone: "App production-ready with CI/CD and docs.")

Each week ends with a demo of working functionality. Track tasks in an issue tracker and review code regularly. Use pull requests and code reviews to maintain quality.

CI/CD, Deployment & Testing Strategies

- **Continuous Integration:** Set up a CI pipeline (e.g. GitHub Actions/Jenkins) that triggers on each push or PR. The pipeline should run unit tests, static analysis (SonarQube/Checkstyle), and build both frontend and backend. For example, use Maven to `mvn test package` for Spring Boot and `npm test && npm run build` for React.
- **Containerization:** Dockerize the app: one image for Spring Boot (running on Tomcat/Netty) and one for the React static build served by a lightweight server (nginx). Push images to a registry (Docker Hub or private registry).
- **Continuous Deployment:** Automate deployment to a hosting platform. Options include Kubernetes (Helm charts), AWS ECS/EKS, or PaaS (Heroku, AWS Elastic Beanstalk). Use `kubectl`/Terraform scripts or cloud CLI in the pipeline to deploy the new image tag. Implement health checks so failed deploys don't go live (roll back on failure).
- **Environments:** Maintain separate environments (dev, staging, prod). Deploy to staging on every main-branch merge; promote to prod on release-tag.
- **Testing:**
- *Unit Tests:* Use JUnit 5 and Mockito for all service and utility logic. Achieve high code coverage.
- *Sliced Tests:* Use `@WebMvcTest` for controller layer validation and JSON format, and `@DataJpaTest` for JPA/repository tests [22]. Use H2 for quick tests or Testcontainers with MySQL for realistic behavior.
- *Integration Tests:* Use `@SpringBootTest` (random port) to load the full app context and test end-to-end flows (calling the API via `TestRestTemplate`) [23]. This catches integration issues.
- *Frontend Tests:* Write unit tests (Jest/React Testing Library) for components and state logic. For e2e testing, use Cypress or Selenium to simulate user actions across the whole app (login, create sprint, etc).
- *Performance Testing:* Run load tests (e.g. with JMeter or k6) to ensure the app meets performance targets under expected user load.
- **Monitoring & Alerts:** In production, use tools like Prometheus+Grafana or a cloud monitoring service to watch key metrics. Configure alerts (e.g. on high error rate or latency).
- **Rollback & Recovery:** Use versioned deployments (Docker tags, Kubernetes replicas) so you can quickly roll back to a previous stable version. Have database migration rollbacks or backups if needed.

- **Documentation:** Maintain a living `README.md` and API docs (Swagger/OpenAPI). Document deployment steps (e.g. infrastructure as code scripts) for future maintainers.

Implementing these practices ensures the system is not only functional but robust, maintainable, and production-ready [7] [23].

Sources: Architecture and stack guidelines from industry best practices [24] [10]; normalization principles [12]; example REST patterns [14]; security tuning [15]; audit logging importance [11]; performance optimizations [25] [8]; CI/CD pipeline steps [7]; testing strategies [22] [23].

---

[1] [2] [4] [6] [24] Spring Boot + React/Next.js: How to Build a Full-Stack App in 2025 - DEV Community
https://dev.to/dev_tips/spring-boot-reactnextjs-how-to-build-a-full-stack-app-in-2025-906

[3] [8] [18] [19] [20] [21] [25] Spring Boot Performance Tuning: 10 Best Practices for High Performance
https://www.javaguides.net/2025/03/spring-boot-performance-tuning.html

[5] Use React and Spring Boot to Build a Simple CRUD App | Okta Developer
https://developer.okta.com/blog/2022/06/17/simple-crud-react-and-spring-boot

[7] Applying CI/CD With Spring Boot | Baeldung
https://www.baeldung.com/spring-boot-ci-cd

[9] [11] Implementing Audit Logs in Spring Boot: A Step-by-Step Guide | by Bhagya Udayangani | Medium
https://medium.com/@pbudayangani9595/implementing-audit-logs-in-spring-boot-a-step-by-step-guide-8d96ad387610

[10] Spring Boot Microservices Architecture Diagram and Design Principles | Sachin Rajput posted on the topic | LinkedIn
https://www.linkedin.com/posts/sachin-rajput-69b4a3204_java-springboot-microservices-activity-7422867668170428416-IcXm

[12] [13] What is Data Normalization and Why Is It Important? - GeeksforGeeks
https://www.geeksforgeeks.org/dbms/what-is-data-normalization-and-why-is-it-important/

[14] Building REST API Using Spring Boot: A Comprehensive Guide | by Pratik T | Medium
https://medium.com/@pratik.941/building-rest-api-using-spring-boot-a-comprehensive-guide-3e9b6d7a8951

[15] [16] [17] Spring Boot Security Best Practices: A Developer's Complete Guide | by Harsh Shah | Medium
https://medium.com/@shahharsh172/spring-boot-security-best-practices-a-developers-complete-guide-e91c49dfd5d3

[22] [23] From Unit to End-to-End Testing with Spring Boot
https://rieckpil.de/spring-boot-testing-from-unit-to-end-to-end-testing/